პროექტის აღწერა

თქვენი მიზანია შექმნათ ხელოვნურ ინტელექტზე დაფუძნებული სისტემა, რომელსაც შეეძლება დიდი მოცულობის დოკუმენტებში ინფორმაციის მოძიება, ანალიზი, ერთმანეთთან დაკავშირება და დასაბუთებული პასუხების გენერირება.

პროექტი არ უნდა წარმოადგენდეს უბრალოდ დოკუმენტებთან სასაუბრო აპლიკაციას (Chat with PDF).

სისტემამ უნდა შეძლოს ისეთი კითხვების დამუშავება, რომლებზეც პასუხის მისაღებად საჭიროა რამდენიმე წყაროს ერთდროული შესწავლა, განსხვავებული ინფორმაციის შედარება და ლოგიკური დასკვნების გამოტანა.

მაგალითი:

ორგანიზაციას აქვს ტექნიკური დოკუმენტაცია, ინციდენტების ანგარიშები, სისტემის ცვლილებების ისტორია და სხვა მასალები.

მომხმარებელი სვამს კითხვას:

„რომელმა სისტემურმა ცვლილებებმა გამოიწვია მარტში გადახდების დამუშავების შეფერხება, რომელი სერვისები დაზიანდა და რომელი დოკუმენტები ადასტურებს ამას?“

სისტემამ უნდა მოიძიოს შესაბამისი ინფორმაცია, დააკავშიროს სხვადასხვა დოკუმენტში არსებული ფაქტები და გასცეს პასუხი წყაროების მითითებით.

თუ ინფორმაცია არასაკმარისია ან ურთიერთგამომრიცხავია, სისტემამ ეს უნდა დააფიქსიროს და არ უნდა შექმნას არარსებული ფაქტები.

2. ძირითადი მოთხოვნები

დოკუმენტების დამუშავება

სისტემას უნდა შეეძლოს სხვადასხვა ფორმატის დოკუმენტების მიღება, დამუშავება და ინდექსირება.

სავალდებულოა:

- მინიმუმ სამი განსხვავებული ფაილის ფორმატის მხარდაჭერა.
- დოკუმენტებიდან ტექსტისა და მნიშვნელოვანი მეტამონაცემების ამოღება.
- ტექსტის ნაწილებად დაყოფა (Chunking).
- დუბლირებული ინფორმაციის დამუშავება.
- ახალი დოკუმენტების დამატება, არსებული დოკუმენტების განახლება და წაშლა.
- მინიმუმ ორი Chunking სტრატეგიის შედარება.

თავად უნდა შეარჩიოთ დამუშავების არქიტექტურა და დაასაბუთოს მიღებული გადაწყვეტილებები.

ვექტორული მონაცემთა ბაზა და ძიება

პროექტში აუცილებელია ვექტორული მონაცემთა ბაზის ან ვექტორული ძიების მხარდაჭერის მქონე მონაცემთა ბაზის გამოყენება.

შესაძლებელია შემდეგი ტექნოლოგიების გამოყენება:

- Qdrant
- Pinecone
- Neon PostgreSQL + pgvector
- Supabase + pgvector
- Weaviate
- Chroma
- ნებისმიერი სხვა შესაბამისი ალტერნატივა

მხოლოდ ჩვეულებრივი SQL ტექსტური ძიება არ ითვლება ამ მოთხოვნის შესრულებად.

სისტემამ უნდა გამოიყენოს Embeddings და Semantic Search.

ამასთან ერთად, აუცილებელია Hybrid Search-ის განხორციელება, რომელიც აერთიანებს სემანტიკურ და საკვანძო სიტყვებზე დაფუძნებულ ძიებას.

თქვენი მხრიდან უნდა განხორციელდეს მინიმუმ ერთი საკუთარი ძიების ან შედეგების დალაგების ალგორითმული კომპონენტი. დაუშვებელია მთლიანი ძიების ლოგიკის მხოლოდ მზა ბიბლიოთეკებზე დაფუძნება.

LLM ინტეგრაცია და RAG

სისტემამ უნდა გამოიყენოს მინიმუმ ერთი Large Language Model.

აუცილებელია:

- Retrieval-Augmented Generation (RAG) არქიტექტურის გამოყენება.
- სხვადასხვა დოკუმენტიდან მიღებული ინფორმაციის გაერთიანება.
- რთული შეკითხვების რამდენიმე ნაბიჯად დამუშავება.
- პასუხების გენერირება შესაბამისი მტკიცებულებების საფუძველზე.
- წყაროებისა და ციტირების მექანიზმის დანერგვა.
- ურთიერთგამომრიცხავი ინფორმაციის აღმოჩენის მცდელობა.
- არასაკმარისი ინფორმაციის შემთხვევაში პასუხისგან თავის შეკავება.
- LLM-თან კომუნიკაციის შეცდომებისა და შეზღუდვების დამუშავება.

სისტემაში უნდა არსებობდეს მინიმუმ ერთი Tool Calling ან ეკვივალენტური სტრუქტურირებული ინსტრუმენტის გამოძახების მექანიზმი.

თავისუფალი ხართ მოდელის, Prompt Engineering მიდგომისა და გამოყენებული Framework-ების არჩევაში.

შედეგების შეფასება

კანდიდატმა უნდა შექმნას საკუთარი ავტომატიზებული შეფასების სისტემა.

მინიმალური მოთხოვნები:

- არანაკლებ 40 სატესტო კითხვა.
- მარტივი და მრავალსაფეხურიანი კითხვები.
- რამდენიმე დოკუმენტის დაკავშირების საჭიროების მქონე კითხვები.
- შეკითხვები, რომლებზეც მონაცემებში პასუხი არ არსებობს.
- ურთიერთგამომრიცხავ ინფორმაციაზე დაფუძნებული კითხვები.
- Retrieval და პასუხების ხარისხის რაოდენობრივი შეფასება.
- მინიმუმ ორი განსხვავებული კონფიგურაციის შედარება.

შეფასებისას გამოყენებული უნდა იყოს შესაბამისი მეტრიკები, მაგალითად Recall@K, MRR, Answer Correctness და Citation Accuracy.

უნდა წარმოადგინოთ არა მხოლოდ შედეგები, არამედ მათი ანალიზიც: სად ცდება სისტემა, რატომ და როგორ შეიძლება გაუმჯობესება.

Backend და მომხმარებლის ინტერფეისი

პროექტს უნდა ჰქონდეს Python-ზე დაფუძნებული Backend და ფუნქციური მომხმარებლის ინტერფეისი.

მომხმარებელს უნდა შეეძლოს დოკუმენტების დამატება, მართვა, კითხვების დასმა და გენერირებული პასუხების წყაროების დათვალიერება.

თავად განსაზღვრეთ API-ის სტრუქტურას, ტექნოლოგიურ არქიტექტურას და ინტერფეისის რეალიზაციის გზას.

პრიორიტეტულია გამართული ფუნქციონალი, კოდის ხარისხი და სისტემის საიმედოობა და არა ვიზუალური დიზაინის სირთულე.

3. მონაცემთა ნაკრები (Dataset)

პროექტის შესასრულებლად უნდა გამოიყენოთ ამ დავალებაზე დართული მონაცემთა ნაკრები.

დასაშვებია დამატებითი საჯარო მონაცემების ან სინთეზური დოკუმენტების გამოყენებაც.

აუცილებელია:

- მინიმუმ 50 000 დოკუმენტის ან დამოუკიდებელი ტექსტური ჩანაწერის დამუშავება.
- რამდენიმე დოკუმენტს შორის დაკავშირებული ინფორმაციის მოძიება.
- წყაროების შენარჩუნება და იდენტიფიცირება.
- არასრული და ურთიერთგამომრიცხავი ინფორმაციის დამუშავება.
- განმეორებადი შეფასების შესაძლებლობა.

არ არის აუცილებელი მთლიანი Dataset-ის გამოყენება, თუ ის ძალიან დიდია. შეგიძლიათ შეარჩიოთ რეპროდუცირებადი ქვენაკრები და დაასაბუთოს არჩევანი.

შეგიძლიათ ამოარჩიოთ თქვენთვის მისაღები ერთი ან რამდენიმე ნაკრები:

https://huggingface.co/datasets/hotpotqa/hotpot_qa

https://huggingface.co/datasets/galileo-ai/ragbench

https://huggingface.co/datasets/xanhho/2WikiMultihopQA

https://github.com/danluu/post-mortems

4. უფასო ჰოსტინგი და Deployment

პროექტი აუცილებლად უნდა იყოს განთავსებული ინტერნეტში და ხელმისაწვდომი საჯარო ბმულით.

უნდა გამოიყენოთ რომელიმე უფასო ჰოსტინგის სერვისი ან სერვისის უფასო ტარიფი.

შესაძლო პლატფორმებია:

- Render
- Vercel
- Hugging Face Spaces
- Koyeb
- Oracle Cloud Always Free
- ნებისმიერი სხვა უფასო ალტერნატივა

ვექტორული მონაცემთა ბაზაც შეიძლება განთავსდეს შესაბამისი პლატფორმის უფასო Cloud ტარიფზე.

სავალდებულოა:

- სისტემის რეალური გაშვება საჯარო URL-ით.
- ჰოსტინგისა და მონაცემთა ბაზის გამართული ინტეგრაცია.
- მონაცემების შენარჩუნება აპლიკაციის გადატვირთვის შემდეგ.
- API გასაღებებისა და სხვა საიდუმლო მონაცემების დაცვა.
- Deployment-ის არქიტექტურის აღწერა.
- ხარჯების, რესურსებისა და უფასო ტარიფის შეზღუდვების გათვალისწინება.

პროექტის შესრულება შესაძლებელი უნდა იყოს ფასიანი ინფრასტრუქტურის შეძენის გარეშე.

უფასო ჰოსტინგის შეზღუდვები, მაგალითად აპლიკაციის დაძინება ან მოთხოვნების ლიმიტი, მისაღებია, თუ ისინი სწორად არის დოკუმენტირებული და არ უშლის ხელს ძირითადი ფუნქციონალის დემონსტრირებას.

5. ტექნიკური ხარისხი

თქვენგან ველოდებით:

- გასაგებ და მოდულარულ Python კოდს.
- პასუხისმგებლობების სწორ გამიჯვნას.
- შეცდომების დამუშავებას.
- ავტომატიზებულ ტესტებს.
- მონაცემთა ბაზასთან და გარე API-ებთან გამართულ ინტეგრაციას.
- გარემოს კონფიგურაციის უსაფრთხო მართვას.
- ტექნიკურ დოკუმენტაციას.
- სისტემის წარმადობისა და LLM ხარჯების გააზრებას.

კონკრეტული ბიბლიოთეკების ან დიზაინის შაბლონების გამოყენება სავალდებულო არ არის.

6. დამატებითი გამოწვევები (Bonus)

დამატებით შეფასდება:

- რამდენიმე LLM-ის ან Embedding მოდელის შედარება.
- Multi-agent ან Agentic RAG არქიტექტურა.
- უფრო განვითარებული Re-ranking მექანიზმი.
- Prompt Injection-ის წინააღმდეგ ტესტირება.
- Caching და წარმადობის ოპტიმიზაცია.
- Streaming პასუხები.
- დეტალური Monitoring და Observability.
- დიდი მოცულობის მონაცემებზე წარმადობის ტესტირება.

დამატებითი ფუნქციონალი არ ანაცვლებს ძირითად მოთხოვნებს.

7. რა უნდა წარმოადგინოთ

დავალების დასრულებისას უნდა წარმოადგინოთ:

1. **GitHub Repository** — სრული კოდი და პროექტის ისტორია.
2. **Live Demo** — გაშვებული აპლიკაციის ბმული.
3. **README** — პროექტის აღწერა, არქიტექტურა, გაშვების ინსტრუქცია და ტექნიკური არჩევანის დასაბუთება.
4. **Evaluation Report** — ტესტირების მეთოდოლოგია, მეტრიკები, შედეგები და ანალიზი.
5. **AI Usage Report** — რომელი AI ინსტრუმენტები გამოიყენა, რაში დაეხმარა და რა შეცვალა დამოუკიდებლად.

8. გასაუბრება და კოდის დაცვა

AI ინსტრუმენტების გამოყენება პროექტის შესრულებისას **ნებადართულია**.

თუმცა, თქვენ სრულად ხართ პასუხისმგებელი წარმოდგენილ კოდზე, მიუხედავად იმისა, საკუთარი ხელით დაწერა თუ AI ინსტრუმენტი დაეხმარა.

ტექნიკური გასაუბრებისას თქვენ მოგიწევთ:

- საკუთარი არქიტექტურის ახსნა.
- კოდის კონკრეტული ნაწილის დეტალურად განხილვა.
- RAG Pipeline-ის მუშაობის ეტაპობრივად აღწერა.
- კოდში შეცდომების აღმოჩენა და გამოსწორება.
- ადგილზე ახალი ფუნქციონალის დამატება ან არსებულის შეცვლა.
- ალგორითმებისა და ტექნიკური გადაწყვეტილებების დასაბუთება.

გასაუბრების პრაქტიკულ ნაწილში AI ასისტენტების გამოყენება დაუშვებელია.

შეფასდება არა მხოლოდ საბოლოო პროდუქტი, არამედ კანდიდატის რეალური ტექნიკური ცოდნა და პრობლემების დამოუკიდებლად გადაჭრის უნარი.